

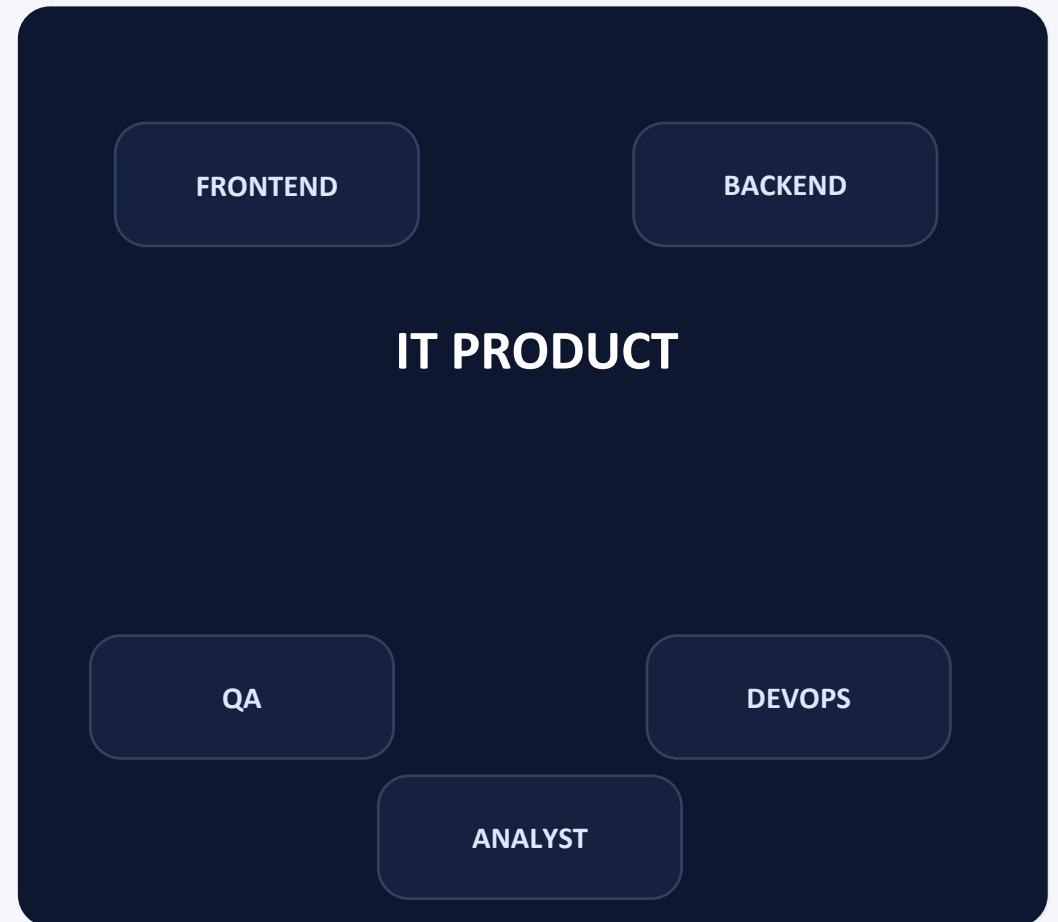
СТАРТ

ЭТАЛОННЫЙ ДЕНЬ

Карта мира IT и язык вакансий

Разберёмся, кто создаёт цифровой продукт, что такое стек и как читать вакансию без паники.

ДЕНЬ 01 / 30



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Не список терминов — конкретные действия.

- 01 Объяснить простыми словами, чем отличаются frontend, backend, QA, analyst и DevOps.
- 02 Увидеть в вакансии три слоя: задачи, требования и стек.
- 03 Прочитать несколько очень простых строк Python и объяснить их обычным языком.
- 04 Не паниковать из-за незнакомых слов: разложить их по категориям и двигаться дальше.

Почему мы начинаем не с синтаксиса

Сначала карта местности — потом детали языка.

Без карты

- Каждое новое слово выглядит отдельной проблемой.
- Вакансия кажется списком из 30 обязательных тем.
- Код воспринимается как набор непонятных символов.

С картой

- Понимаешь, кто за что отвечает.
- Видишь структуру вакансии.
- Незнакомое слово можно отложить в правильную «полку».

Главная цель дня: перестать видеть IT как одну огромную неизвестность.

АНАЛОГИЯ

IT-команда похожа на ресторан

Пользователь видит результат. Внутри — несколько разных ролей.

ЗАКАЗ

Пользователь говорит,
что ему нужно.

Потребность

КУХНЯ

Команда превращает
запрос в работающий
результат.

Разработка

КОНТРОЛЬ

Кто-то проверяет
качество и сценарии
ошибок.

QA

ДОСТАВКА

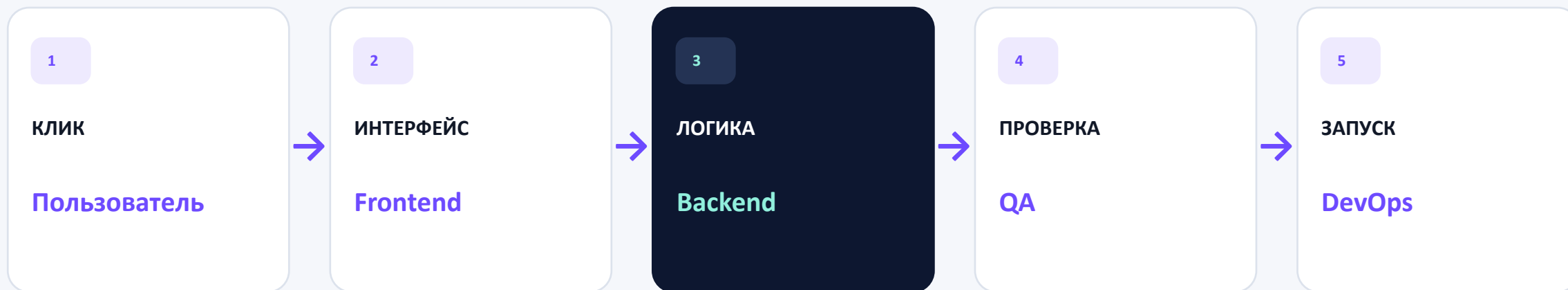
Результат должен
стабильно прийти до
пользователя.

DevOps

Аналогия помогает увидеть роли, но дальше мы свяжем их с реальным цифровым продуктом.

Один клик пользователя запускает цепочку

Посмотрим на кнопку «Оформить заказ» как на общую задачу команды.

**Важно**

Одна реальная задача часто проходит через несколько ролей. Границы между ними зависят от команды.

РОЛИ В IT

FRONTEND

То, что ближе к интерфейсу пользователя

ПРОСТЫМИ СЛОВАМИ

**Кнопки, формы, страницы,
поведение в браузере.**

ПРИМЕР В ПРОДУКТЕ

**Изменить, что происходит после нажатия кнопки
«Войти».**

Это направление разработки.

РОЛИ В IT

BACKEND

Логика на сервере и работа с данными

ПРОСТЫМИ СЛОВАМИ

**Проверки, расчёты, запросы
к данным, ответы
приложению.**

ПРИМЕР В ПРОДУКТЕ

Проверить пароль и создать сессию пользователя.

Это направление разработки.

РОЛИ В IT

QA

Качество и проверка сценариев

ПРОСТЫМИ СЛОВАМИ

Ищет проблемы, проверяет ожидания и условия возникновения ошибки.

ПРИМЕР В ПРОДУКТЕ

Проверить вход с верным, неверным и пустым паролем.

Это роль рядом с разработкой, которая помогает продукту работать как задумано.

РОЛИ В IT

ANALYST

Переводит желание в понятные требования

ПРОСТЫМИ СЛОВАМИ

Уточняет сценарии, правила и крайние случаи.

ПРИМЕР В ПРОДУКТЕ

Что показывать после 5 неверных попыток входа?

Это роль рядом с разработкой, которая помогает продукту работать как задумано.

DEVOPS

Доставка изменений и рабочая среда

ПРОСТЫМИ СЛОВАМИ

Связывает разработку, инфраструктуру и надёжный запуск.

ПРИМЕР В ПРОДУКТЕ

Доставить новую версию сервиса в рабочую среду.

Это роль рядом с разработкой, которая помогает продукту работать как задумано.

PROJECT MANAGER

Организация движения работы

ПРОСТЫМИ СЛОВАМИ

Помогает с договорённостями, зависимостями и сроками.

ПРИМЕР В ПРОДУКТЕ

Согласовать, когда frontend и backend смогут завершить связанную задачу.

Это роль рядом с разработкой, которая помогает продукту работать как задумано.

ПРОВЕРКА 1

Кто скорее всего возьмёт эту задачу?

Сначала ответь сам. Потом сверимся.

«Нужно проверить, что форма регистрации показывает ошибку при пустом email и не создаёт пользователя.»

A Frontend

B Backend

C QA

D DevOps

Не ищи “идеальную профессию”. Спроси: кто здесь прежде всего проверяет ожидаемое поведение?

ОТВЕТ

Ответ: QA

Но реальная задача может затронуть несколько ролей.

ПОЧЕМУ

В формулировке главное действие — проверить несколько сценариев и ожидаемый результат.

КТО ЕЩЁ МОЖЕТ УЧАСТВОВАТЬ

- Frontend — показывает ошибку в интерфейсе.
- Backend — проверяет данные и решает, создавать ли пользователя.
- QA — проверяет сценарии и фиксирует проблему.

Роли не живут по отдельности

Один продукт объединяет работу нескольких специалистов.



Project manager помогает организовать движение задач между участниками.

Стек = набор инструментов проекта

Не “уровень программиста” и не “список сотрудников”.

Python

язык

Flask

веб-инструмент

PostgreSQL

данные

Git

история кода

Сегодня достаточно понимать роль каждого слова. Глубоко изучать Flask или PostgreSQL пока не нужно.

СТЕК

Читаем строку стека слоями

Каждое слово отвечает на свой маленький вопрос.

Python

На чём пишут основную логику?

Язык программирования

Flask

Чем собирают веб-часть backend?

Веб-фреймворк

PostgreSQL

Где хранят структурированные данные?

База данных

Git

Как хранят историю изменений кода?

Контроль версий

Python + Flask + PostgreSQL + Git

СТЕК

Стек ≠ требования

Списки могут пересекаться, но отвечают на разные вопросы.

СТЕК

Чем пользуется проект?

Python • Flask • PostgreSQL • Git

ТРЕБОВАНИЯ

Что ожидают от кандидата?

Основы Python • Git • базовый SQL

Один и тот же термин может встретиться в обоих блоках — это нормально.

Код не магия: одна команда

Сегодня мы только учимся читать запись. Запоминать синтаксис не нужно.



day01_step1.py

```
1 print("Backend developer")
```

Человеческий перевод

**«Покажи на экране текст
Backend developer».**

РЕЗУЛЬТАТ

Backend developer

Разбираем каждый символ

Ничего “очевидного” для новичка.



day01_step1.py

```
1 print("Backend developer")
```



print

имя команды: вывести что-то на экран



()

скобки: внутри лежат данные для команды



" ... "

кавычки: Python видит, что внутри обычный текст



Связь

Код → Python выполняет команду → текст появляется на экране.

БЫЛО → ИЗМЕНИЛИ

Меняем данные — меняется результат

Команда та же. Текст внутри другой.

БЫЛО

```
before.py
1 print("Backend developer")
```

РЕЗУЛЬТАТ

Backend developer

ИЗМЕНИЛИ

```
after.py
1 print("QA engineer")
```

РЕЗУЛЬТАТ

QA engineer

Мы не трогали `print()`. Изменили только текст, который просим вывести.

Программа может хранить найденную информацию

Это только предварительный взгляд на переменную. Полноценно — позже.



day01_step2.py

```
1 role = "Backend developer"  
2 print(role)
```

role

понятное имя для данных

=

«сохрани значение справа под именем слева»

**"Backend
developer"**

текстовое значение

print(role)

выведи то, что хранится в role

РЕЗУЛЬТАТ

Backend developer

Сохраним стек одной строкой

Связываем термин дня с простым кодом.



day01_stack.py

```
1 stack = "Python + Flask + PostgreSQL"  
2 print(stack)
```

Объясни своими словами

«Сохранили строку со стеком под именем stack, потом вывели её на экран».

РЕЗУЛЬТАТ

Python + Flask + PostgreSQL

Что появится на экране?

Не запускай код. Сначала предскажи результат.



question.py

```
1 role = "QA"  
2 print(role)
```

ВОПРОС

Какой текст появится после запуска?

Подсказка: найди, что сейчас хранится в role.

ОТВЕТ

Ответ: QA

Читаем сверху вниз и следим за значением.

1

role = "QA"

В role сохраняется текст QA.

2

print(role)

Команда просит вывести содержимое role.

3

QA

Именно этот текст появляется на экране.

Если можешь объяснить эти три шага без подсказки — код уже не выглядит случайным набором символов.

Ошибка — это сообщение, а не приговор

Покажем одну типичную синтаксическую ошибку.



wrong.py

```
1 print("Backend developer)
```

Что сломано?

Открывающая кавычка есть, закрывающей — нет.



fixed.py

```
1 print("Backend developer")
```

Человеческий перевод ошибки

«Я не понимаю, где заканчивается текст.
Проверь запись».

Вакансия — это не экзаменационный билет

Это описание роли. Научимся видеть её структуру.

1

РОЛЬ

Кого ищут?

2

ПРОДУКТ

Что делает команда?

3

ЗАДАЧИ

Что человек будет делать?

4

ТРЕБОВАНИЯ

Что ожидают знать?

5

СТЕК

Какими технологиями пользуются?



Сегодня мы учимся видеть эти 5 блоков — не оценивать себя по каждому пункту.

Must-have и nice-to-have

Слова-маркеры помогают понять силу требования.

MUST-HAVE

Компания считает это необходимым для роли.

Примеры маркеров: «обязательно», «требуется», «необходимо».

NICE-TO-HAVE

Полезное преимущество, но не всегда обязательное.

Маркеры: «будет плюсом», «желательно», «преимущество».

Формулировки компаний отличаются. Поэтому всегда читай контекст, а не одно слово.

Сначала решим одну вакансию вместе

После этого ты получишь очень похожую самостоятельную задачу.

УЧЕБНАЯ ВАКАНСИЯ

Junior Python Backend Developer

Продукт: сервис бронирования спортивных площадок.

Задачи добавлять небольшие backend-функции • исправлять ошибки • работать с данными

Ожидаем основы Python • основы Git • базовое понимание SQL • готовность задавать вопросы

Стек **Python • Flask • PostgreSQL • Git**

Не пытайся сейчас “знать Flask”. Наша задача — классифицировать информацию.

Как рассуждать: 3 вопроса

Превращаем длинный текст вакансии в понятную схему.

1

Что человек будет ДЕЛАТЬ?

Ищем глаголы и рабочие действия.

2

Что от него ожидают ЗНАТЬ?

Ищем знания, навыки, опыт.

3

ЧЕМ пользуется проект?

Ищем языки, фреймворки, базы, инструменты.

Эти три вопроса — твой “фильтр” для первого чтения вакансии.

Разложили вакансию по полкам

Вот что получилось после трёх вопросов.

ЗАДАЧИ

- добавлять backend-функции
- исправлять ошибки
- работать с данными

ТРЕБОВАНИЯ

- основы Python
- основы Git
- базовый SQL

СТЕК

- Python
- Flask
- PostgreSQL
- Git

Теперь текст вакансии уже не выглядит одной стеной из терминов.

Переводим сложную фразу на человеческий язык

Смысл важнее формулировки.

«Участие в code review»



Смотреть изменения в коде вместе с другими и получать обратную связь.

«Базовый SQL»



Уметь выполнять простые действия с данными через запросы.

«Docker будет плюсом»



Полезное дополнительное знание; по формулировке — не главное требование.

Четыре ловушки при чтении вакансии

Замечаем их заранее — и не тратим силы зря.

01

«Нужно знать каждое слово»

Сначала классифицируй, потом изучай по приоритету.

02

«Стек = требования»

Стек описывает инструменты проекта.

03

«Junior должен уметь всё»

Читай конкретные задачи и формулировки вакансии.

04

«Незнакомое = слишком сложно»

Незнакомое слово — просто точка для будущего изучения.

ПРАКТИКА 1

Определи главную роль

Похожий пример мы уже разобрали. Теперь твоя попытка.

«Нужно изменить поведение кнопки “Добавить в корзину” и показать рядом счётчик товаров».

A

Frontend

B

Backend

C

QA

D

Analyst

Сначала выбери один ответ. Потом объясни, какое слово в условии дало тебе подсказку.

Основной ответ: Frontend

Потому что речь о видимом поведении интерфейса.

КЛЮЧЕВАЯ ПОДСКАЗКА

«Показать рядом счётчик» — это часть интерфейса, которую видит пользователь.

Но задача может быть шире

Backend может обновлять корзину, а QA — проверять сценарии. Мы выбираем роль по главному действию в формулировке.

Хороший ответ объясняет не только “кто”, но и “почему”.

Разложи строки по трём категориям

Задачи • требования • стек

1

Исправлять небольшие ошибки в API

2

Python, Flask, PostgreSQL, Git

3

Понимание основ Python

4

Добавлять backend-функции

5

Базовое знание SQL

6

Участие в code review

Запиши рядом с каждым номером одну букву: З = задача, Т = требование, С = стек.

Ответ: 1–3, 2–С, 3–Т, 4–3, 5–Т, 6–3

Проверим логику, а не только буквы.

ЗАДАЧИ

1. Исправлять ошибки
4. Добавлять функции
6. Участвовать в code review

ТРЕБОВАНИЯ

3. Основы Python
5. Базовое знание SQL

СТЕК

2. Python, Flask, PostgreSQL, Git

Глаголы часто подсказывают задачи. Названия технологий часто подсказывают стек.

САМОСТОЯТЕЛЬНО

Собери карточку вакансии

Это главная самостоятельная практика дня.

РОЛЬ

Кого ищут?

ЗАДАЧИ

3–5 действий

ТРЕБОВАНИЯ

3–5 знаний/навыков

СТЕК

Технологии проекта

НЕЗНАКОМОЕ

3 слова — только отметить



Не оценивай себя. Задача — увидеть структуру текста.

Застрял? Открывай подсказки по очереди

Не читай все сразу — дай себе шанс подумать.

1

Найди глаголы

«добавлять», «исправлять», «участвовать» часто указывают на задачи.

2

Найди “знать / понимать / опыт” Это обычно ведёт к требованиям.

3

Собери имена технологий

Python, Git, SQL, Flask и похожие слова часто относятся к стеку.

Как может выглядеть хороший результат

Формат важнее “красивых слов”.

РОЛЬ

Junior Python Backend Developer

ЗАДАЧИ

добавлять функции • исправлять ошибки • работать с данными

ТРЕБОВАНИЯ

основы Python • Git • базовый SQL

СТЕК

Python • Flask • PostgreSQL • Git

НЕЗНАКОМОЕ

Flask → технология • PostgreSQL → технология • code review → рабочий процесс

Ключевой критерий: ты можешь объяснить, почему каждая строка попала именно в эту категорию.

Запишем один вывод в код

Не для автоматизации вакансии — только чтобы связать смысл и Python.



vacancy_note.py

```
1 role = "Backend developer"
2 stack = "Python + Flask + PostgreSQL"
3 print(role)
4 print(stack)
```

РЕЗУЛЬТАТ

Backend developer
Python + Flask + PostgreSQL

Что здесь нового?

Две переменные хранят два разных вывода из вакансии. Потом мы выводим каждую.

Найди две ошибки

Сравни с рабочим маленьким примером.

find_errors.py

```
1 role = "Backend developer"
2 print(role
```

fixed.py

```
1 role = "Backend developer"
2 print(role)
```

Симптом

Python не может корректно разобрать запись.

Проверь парные символы: кавычки и скобки.

Правильно

Закружили кавычку и скобку.

МИНИ-ТЕСТ

Проверь себя: 5 вопросов

Ответь коротко — без подсказок.

1. Что такое стек?

2. Кто чаще всего работает с интерфейсом?

3. Чем задача вакансии отличается от требования?

4. Что выведет: `role = "QA" → print(role)`?

5. Что сделать с незнакомым словом в вакансии?

Сформулируй ответы своими словами. Термин считается понятым, когда ты можешь объяснить его без заученной фразы.

Ответы и почему они такие

Проверяем понимание, а не память формулировок.

- 1 Набор технологий проекта.
- 2 Frontend — если речь о пользовательском интерфейсе.
- 3 **Задача = действие на работе; требование = знание/навык кандидата.**
- 4 QA — именно это значение хранится в role.
- 5 Отметить, определить категорию по контексту и не пытаться изучить всё сразу.

❏ Ошибся? Вернись к конкретному примеру, а не перечитывай весь урок.

ИТОГ

Теперь ты умеешь больше, чем “знаешь термины”

Проверим реальные способности после Дня 1.



объяснить роли IT через один продуктовый сценарий



отличить frontend от backend на базовом уровне



объяснить, что такое технологический стек



разделить вакансию на задачи, требования и стек



прочитать 2–4 простые строки Python и объяснить их

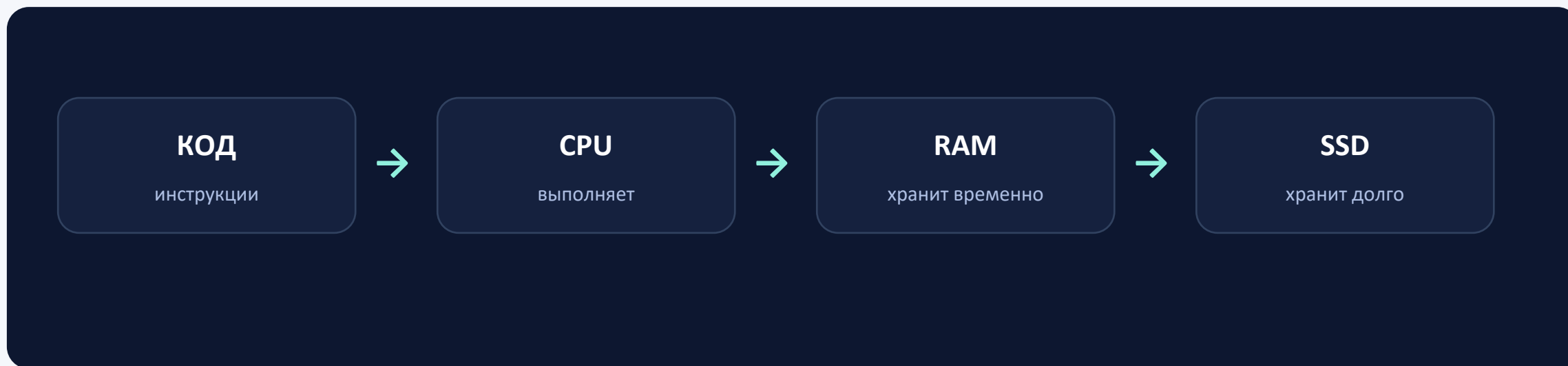


спокойно отнестись к простой синтаксической ошибке

ЗАВТРА

Следующий вопрос: где вообще работает код?

День 2 — как устроен компьютер.



Сегодня: кто создаёт продукт и как читать язык вакансий.

Завтра: где физически выполняются команды программы и чем память отличается от диска.